identical testing environments

from laptop to ci
with tmt and testing farm

•••

cristian le & petr šplíchal

we

- petr šplíchal
  - psss / psplicha@redhat.com
  - senior principal software quality engineer
- cristian le
  - lecris / (don’t email me)
  - senior software engineer
- what are we doing?
  - packaging & testing experience
  - improving testing tools & processes
  - testing the operating system

you

- are exposed to tmt and testing-farm
- maintain some tmt tests
- just curious how the bread is made

not you

- do not know about tmt yet or want to get started
  - guide: tmt.readthedocs.io/en/stable/guide.html
  - workshop: github.com/teemtee/workshop (wip, check back later)
- do not run tmt against artifacts (koji, copr, etc.) or you download manually
  - (but maybe you should?)

agenda

- why is it so difficult?
- let’s make it easy!
- how do we do it?
- are we there yet?

vocabulary

- testing farm ... testing system as a service (running tests at scale)
- tmt ... test management tool (specification & execution)
- tmt run ... a single execution of tests/plans
- tmt steps ... test execution stages
  - discover
  - provision
  - prepare
  - execute
  - finish
  - report
  - cleanup

why is it so difficult?

the problem: tests behave differently in ci vs. laptop

- ci × laptop
  - there was a failure or error in the ci
  - cannot reproduce it locally, it just works!
- ci × laptop
  - everything green when run in the ci
  - failures or errors when run locally
  - or unable to execute at all

root causes: hidden differences

- guest setup
  - repositories installed or not
  - repositories enabled or not
  - various buildroot tweaks
  - custom scripts / workarounds
- package installation
  - different implementation
  - custom handling of conflicting packages

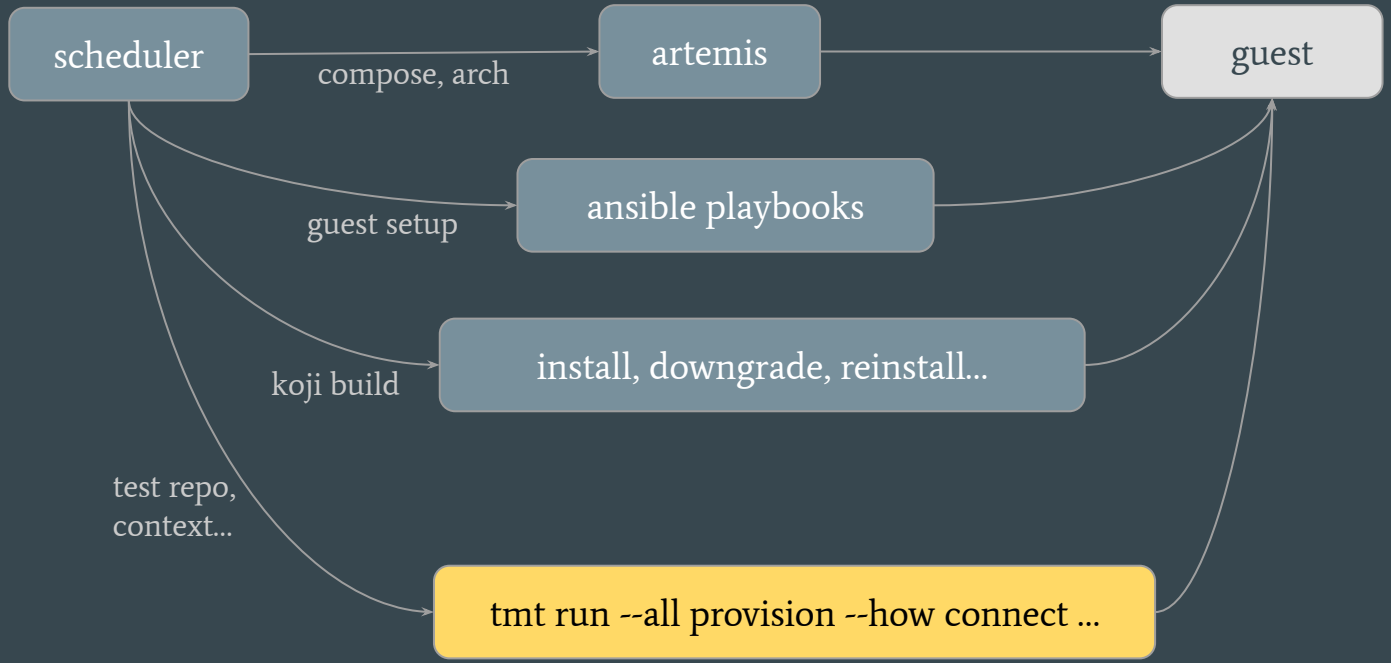
let’s make it easy!

vision: identical behavior across all environments

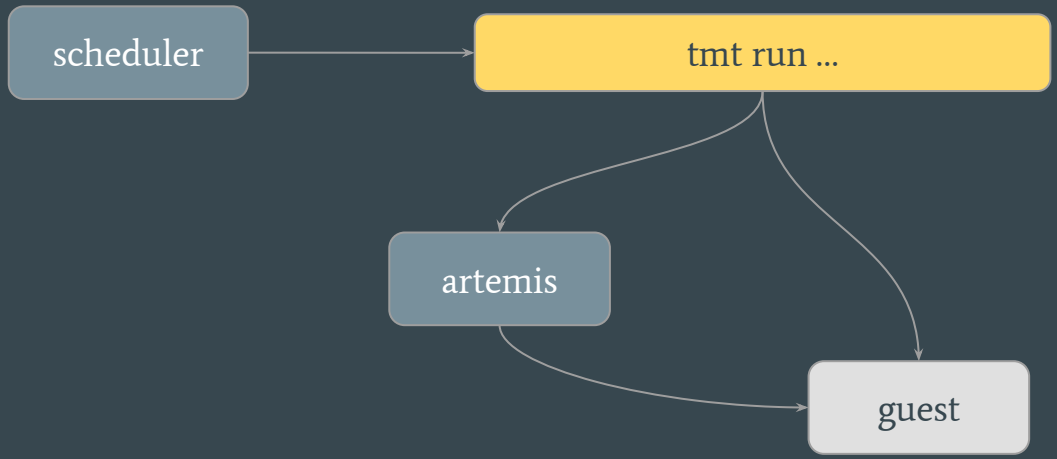
- guest profiles (available at standard location)
- tmt policies (enable consistent, standard setup)
- artifact install (single way to install packages)
- one pipeline (drop the old to lower maintenance)

```
# select policy, provide artifact
tmt run \
    --policy-name fedora-ci
    prepare --insert --how artifact --provide koji.task:141223362
```

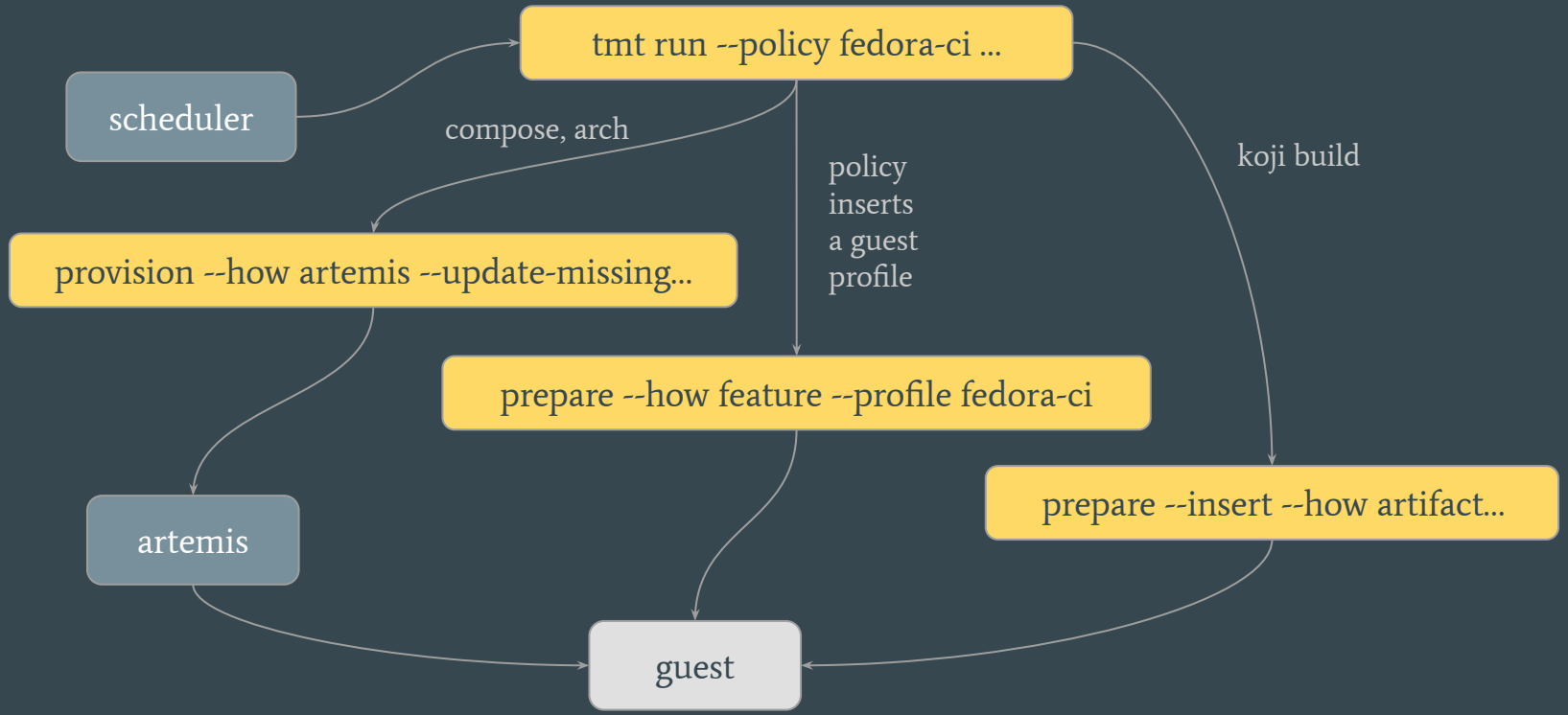
testing farm: old pipeline



testing farm: new pipeline



testing farm: new pipeline



reproducer: before

```
git clone ...
cd testcode

curl -o guest-setup-0.sh -L https://artifacts.dev.testing-farm.io/.../install_commands.sh
curl -LO https://artifacts.dev.testing-farm.io/.../tmt-environment-plans-simple.yaml

tmt run --until provision --environment @tmt-environment-plans-simple.yaml \
    plan --name ^/plans/simple$ \
    provision --how virtual --image Fedora-44

tmt run --last login < guest-setup-0.sh
tmt run --last --since prepare
```

← Lots of magic here

reproducer: after

```
git clone ...
cd testcode

tmt run --all \
    --policy-name fedora-ci \
    --environment @https://.../tmt-environment-plans-simple.yaml \
    plan --name '^/plans/simple$' \
    prepare --insert --how artifact --provide koji.task:141223362 \
    provision --how virtual --image Fedora-44
```

how do we do it?

tmt policy: how it works

- policy := jinja yaml file
- replace test/plan content with rendered jinja
  - enable checks
  - apply guest profile
  - override provision method
  - ...
- enforce test environment and standard (policy)
- examples: rhel-ci and fedora-ci

```
test-policy:
  - check: |
      {% macro emit_original() %}
          {% for check in VALUE %}
      - {{ check | to_yaml | indent(2, first=False) }}
          {% endfor %}
      {% endmacro %}

      {% if 'avc' in VALUE | map(attribute='how') %}
      {{ emit_original() }}

      {% else %}
      - how: avc
        result: respect

      {{ emit_original() }}
      {% endif %}
```

tmt policy: examples

```
test-policy:
  check: |
    - how: avc
      result: respect
    {{ VALUE | list | to_yaml }}

plan-policy:
  prepare: |
    - how: feature
      epel: enabled
      order: 31
    {{ VALUE | list | to_yaml }}
```

← Any tmt test field

← Add a new check

← Add original content

← Enable epel before installation

environment profiles: how it works

- ci specific steps are defined in the profile/policy
- profiles: an ansible playbook managed by ci provider
  - add a (buildroot) repository, enable epel, etc.
  - all testing-farm workarounds
- policies: a (complex) way to edit plans/tests
  - inject a check to all tests, e.g. avc
  - add a prepare phase, e.g. profile

environment profiles: examples

(Provisional)

```
curl -LO https://gitlab.com/testing-farm/profiles/-/raw/main/policies/rhel-ci.yaml
tmt run --all \
     --policy-file rhel-ci.yaml \
     ...
```

artifact install: how it works

- prepare repository
  - download all rpms
  - create a dnf repository with all the rpms and enable
  - set priority higher than system
- identify required/recommended packages
  - install them on the guest (specific rpms)
  - verify they are coming from the desired repositories

artifact install: examples

```
# test specific koji build on your laptop
tmt run -vvv --all \
    provision --how virtual --image fedora-44 \
    prepare --how artifact --provide koji.task:141223362

# do the same in testing farm
testing-farm request \
    --compose Fedora-44 \
    --fedora-koji-build 141223362 \
    --pipeline-type tmt-multihost
```

multihost pipeline: how it all works together

- skip all workarounds testing-farm had
- move the workarounds to profiles
- orchestrators (packit, fedora-ci, etc.) opt-in to specific profile
- add artifacts via tmt prepare

multihost pipeline: full example

(Provisional)

```
testing-farm request \
    --git-url ... --git-ref ... \
    --pipeline-type tmt-multihost \
    (-T TMT_POLICY_NAME=rhel-ci) \
    --fedora-koji-build 141223362
```

are we there yet?

status: where we are

- guest profiles implemented
  - ansible playbooks migrated
- artifact install implemented
  - essential functionality should be working
  - enabled in the multihost pipeline
- initial policies outlined
  - rhel-ci
  - fedora-ci

next steps: what's missing

- making multi-host pipeline the default
  - internal testing and comparing with old approach
  - Fedora change proposal
- distributing environment variables (instead of downloading policy)
- moving all workarounds to appropriate profile
- tmt artifact installation may fail in some cases

future reproducer: using recipes?

```
tmt run \
    --recipe https://artifacts.dev.testing-farm.io/.../recipe.yaml \
    test --name /my/failed/test
```

questions?

links

- docs
  - tmt.readthedocs.io
  - fmf.readthedocs.io
  - docs.testing-farm.io
  - packit.dev
- talks
  - enjoy creating, executing and enabling tests using tmt — nest with fedora 2020
  - easily enable tests in fedora, rhel, github — devconf.cz 2021
  - tmt = freedom for tests + comfort for users — devconf.cz 2022
  - tmt - test management tool — brněnské pyvo 2023
  - testing farm test environment setup profiles — flock 2025
  - what's cooking in copr, testing farm, tmt, packit and log detective — flock 2026

thanks!